

## Горчак Дмитрий, 212. Отчёт: локальный репозиторий git и задача о фигурах на плоскости

Отчёт о выполненной работе: установка git, создание локального репозитория, работа с ним и текущее состояние программы. Скриншоты сняты с окон MSYS2 на моём компьютере.

Сделано:

- 1) git установлен в MSYS2 и настроен, вся работа сохраняется через локальный репозиторий ~/cones;
- 2) готова иерархия классов Point -> Circle -> Cone; фигура создаётся по количеству чисел в строке файла: два числа – точка, три – круг, четыре – конус;
- 3) площадь основания и объём реализованы методами, а не полями; поля закрыты, доступ через get/set;
- 4) объём считается отдельно, по запросу: номер нужной фигуры передаётся программе аргументом командной строки;
- 5) поле фигур нарисовано в gnuplot: вид сверху, карта высот, ландшафт и объёмный рисунок точками по поверхности фигур;
- 6) фигуры складываются в двунаправленный список STL и упорядочиваются по площади основания и по объёму;
- 7) вокруг конусов рассыпаны гауссовы точки, разбиты на блоки сетки и собраны в кластеры двумя независимыми алгоритмами;
- 8) отдельной программой проверен датчик случайных чисел: одномерное и двумерное нормальное распределение;
- 9) написаны четыре эссе – по git, по STL и двунаправленному списку, по гауссову распределению и по кластерному анализу.

Уточнение по фигуре: преподаватель подтвердил, что фигура с круглым основанием и высотой – это конус, а не пирамида. Поэтому класс называется Cone, а объём считается по формуле  $V = S * h / 3 = \pi * r * r * h / 3$ , где  $S$  – площадь основания, взятая у базового класса Circle. Для сравнения в классе оставлен метод volume\_ngon(n) – объём пирамиды с правильным n-угольником в основании, вписанным в тот же круг: при росте n он стремится к объёму конуса.

### 1. Установка git в MSYS2

Git поставлен штатным менеджером пакетов MSYS2 командой `pacman -S git`. На скриншоте – запись в журнале установки, версия пакета, версия самой программы, путь к ней и начало списка команд из встроенной справки.

```
wasdm@ziliboba MINGW64 ~/cones
$ grep -m1 'installed git' /var/log/pacman.log
[2026-09-17T18:16:00+0300] [ALPM] installed git (2.55.0-1)

wasdm@ziliboba MINGW64 ~/cones
$ pacman -Q git
git 2.55.0-1

wasdm@ziliboba MINGW64 ~/cones
$ git --version
git version 2.55.0

wasdm@ziliboba MINGW64 ~/cones
$ which git
/usr/bin/git

wasdm@ziliboba MINGW64 ~/cones
$ git help -a | head -8
See 'git help <command>' to read about a specific subcommand

Main Porcelain Commands
add                Add file contents to the index
am                Apply a series of patches from a mailbox
archive           Create an archive of files from a named tree
backfill          Download missing objects in a partial clone
bisect            Use binary search to find the commit that introduced a bug

wasdm@ziliboba MINGW64 ~/cones
$
```

Скриншот 1. Установка git и проверка: `pacman -Q git`, `git --version`, `git help -a`

## 2. Локальный репозиторий

Репозиторий создан прямо в папке с задачей, история ведётся по шагам работы: отдельный коммит на каждый класс, на ввод-вывод, на программу и на скрипт gnuplot.

```
$ cd ~/cones
$ git init -b main
$ git config user.name "Gorchak Dmitriy"
$ git config core.autocrlf false
```

Работа над списком из STL велась в отдельной ветке `stl-list` и влита слиянием с ключом `--no-ff`, поэтому в истории видно, что это была отдельная задача. На коммиты, соответствующие срокам, поставлены метки: `stage-19-09` — классы и список, `stage-26-09` — точки, сетка блоков, кластеры и эссе.

```
wasdm@ziliboba MINGW64 ~/cones
$ ls
README.md  cluster.cpp  figures.txt  gauss_demo.cpp  list_demo.cpp  plot_gauss.gp  txt
bin         cluster.h   figures_io.cpp  grid.cpp        main.cpp       plot_points.gp
build.sh   cone.cpp   figures_io.h   grid.h          make_txt.sh    point.cpp
circle.cpp cone.h     gauss.cpp     grid_point.cpp  out            point.h
circle.h   essays    gauss.h       grid_point.h    plot_figures.gp points_demo.cpp

wasdm@ziliboba MINGW64 ~/cones
$ git config --list --local | grep -E 'user|autocrlf' | sed 's/\\(user.email=\\).*/\\1***@***/'
core.autocrlf=false
user.name=Gorchak Dmitriy
user.email=***@***

wasdm@ziliboba MINGW64 ~/cones
$ git log --oneline --decorate | head -6
061a0d1 (HEAD -> main) Правки формулировок в эссе по STL и кластерам
e3c9478 Шесть двумерных облаков вместо двух: разбросы и связь координат по отдельности
8f6ef9d Отчёт: обновлённый снимок истории репозитория и метка stage-26-09
4bbf8f5 (tag: stage-26-09) Эссе по гауссову распределению и кластерам, скриншоты запусков 05-08
5a78e83 Скрипты gnuplot для кластеров и распределения, сборка четырёх программ
3e07a8d Датчик нормальных чисел и программа проверки

wasdm@ziliboba MINGW64 ~/cones
$ git status -sb
## main
M essays/screens/01-install-git.png
M essays/screens/02-repository-log.png

wasdm@ziliboba MINGW64 ~/cones
$
```

Скриншот 2. Состав папки, локальные настройки, история коммитов и чистое рабочее дерево

Все исходники лежат в одной папке, без вложенных каталогов: так их проще найти и отправить. Собранные программы складываются в `bin`, всё, что создают программа и `gnuplot`, — в `out`; обе папки перечислены в `.gitignore` и в историю не попадают, поэтому `git status` показывает чистое дерево сразу после сборки и запуска.

### 3. Рабочий цикл: правка, сравнение, фиксация

Обычная работа с локальным хранилищем выглядит так: изменить файл, посмотреть, что именно изменилось, подготовить изменение к фиксации и зафиксировать его с понятным сообщением. На скриншоте в файл данных добавлена ещё одна точка — строка из двух чисел; команда `sed` вставляет её сразу в нужный раздел файла.

```
~/cones

wasdm@ziliboba MINGW64 ~/cones
$ git diff
diff --git a/essays/screens/01-install-git.png b/essays/screens/01-install-git.png
index 1318be7..bf263ed 100644
Binary files a/essays/screens/01-install-git.png and b/essays/screens/01-install-git.png differ
diff --git a/essays/screens/02-repository-log.png b/essays/screens/02-repository-log.png
index 4b69ba6..e202f57 100644
Binary files a/essays/screens/02-repository-log.png and b/essays/screens/02-repository-log.png differ
diff --git a/figures.txt b/figures.txt
index ff7957a..29b840e 100644
--- a/figures.txt
+++ b/figures.txt
@@ -33,6 +33,7 @@
 7.5 10.5
 13.0 7.0
 20.0 16.0
+ 4.0 16.0

# строка с ошибкой: радиус отрицательный -- объект создан не будет
10.0 10.0 -1.0 5.0

wasdm@ziliboba MINGW64 ~/cones
$ git add figures.txt

wasdm@ziliboba MINGW64 ~/cones
$ git commit -m 'Ещё одна точка в исходных данных'
[main 1c826a6] Ещё одна точка в исходных данных
1 file changed, 1 insertion(+)

wasdm@ziliboba MINGW64 ~/cones
$ git log --oneline -3
1c826a6 (HEAD -> main) Ещё одна точка в исходных данных
061a0d1 Правки формулировок в эссе по STL и кластерам
e3c9478 Шесть двумерных облаков вместо двух: разбросы и связь координат по отдельности

wasdm@ziliboba MINGW64 ~/cones
$
```

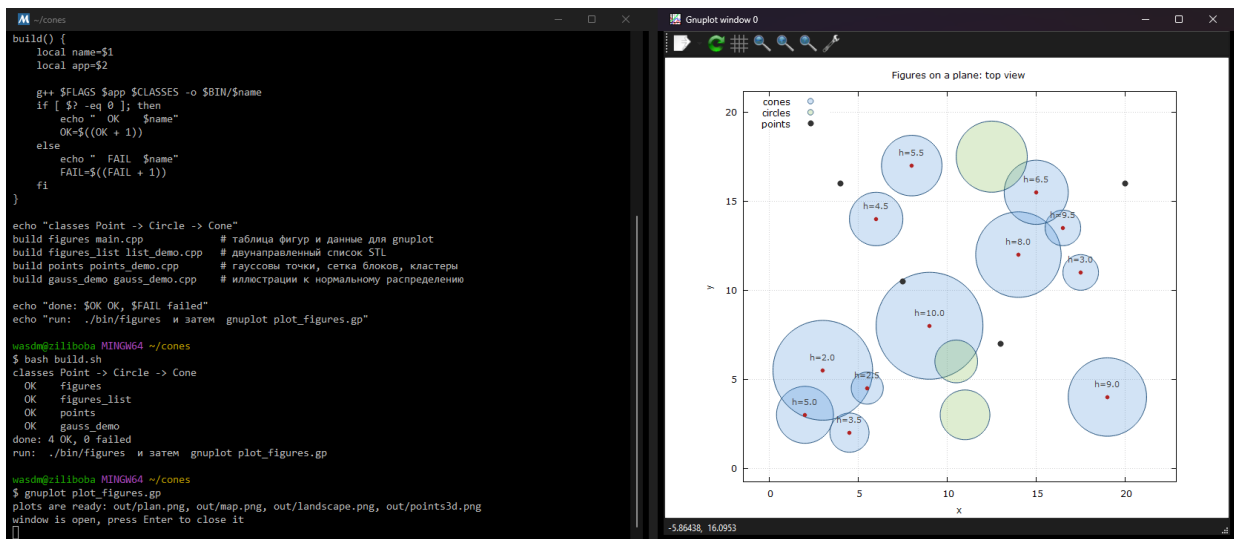
Скриншот 3. git status, git diff, git add, git commit и история после фиксации

Видно главное свойство diff: git показывает не файл целиком, а одну добавленную строку с отметкой + и несколько строк вокруг неё для понимания места.

## 4. Сборка, запуск и построение рисунков

Программы собираются одной командой `bash build.sh`. Сам скрипт показан на скриншоте командой `cat`: внутри него вызовы компилятора `g++` с перечислением файлов классов и ключом `-o`, задающим имя готовой программы; по коду завершения `g++` скрипт печатает OK или FAIL. Готовые программы складываются в папку `bin`, а всё, что они создают, — в папку `out`.

Скрипт `plot_figures.gr` строит по этим данным рисунки в файлы `png` и дополнительно открывает окно `gnuplot` с видом сверху — оно видно справа от терминала. Окно держится командой `pause`, пока в терминале не нажать `Enter`.



Скриншот 4. Текст скрипта сборки, сборка четырёх программ и окно gnuplot рядом с терминалом

Границы поля по  $x$  и  $y$  считает сама программа: она находит крайние точки оснований, добавляет запас и записывает готовые команды gnuplot в файл `out/range.gp`, который скрипт подключает командой `load`. Поэтому рисунок подстраивается под данные, а в скриптах нет ни одного числа, заданного вручную.

## 5. Успешные запуски программ

Собираются четыре программы, и каждая запускается отдельно. Первая – `figures`: она читает файл данных, печатает таблицу фигур и готовит данные для gnuplot. Две строки в файле сделаны намеренно неправильными (отрицательный радиус и одно число в строке); программа сообщает о них и продолжает работу, поэтому из двадцати одной строки получается девятнадцать объектов.

```
~ /cones
wasdm@ziliboba MINGW64 ~/cones
$ ./bin/figures | sed -n '14,40p'
  get_r() = 2.00, get_h() = 6.00, volume() = 25.13
  after set_r(4): volume() = 100.53 (the value is not stored, it is computed)
  rejected: radius must be >= 0
  rejected: height must be >= 0

--- Reading properties from 'figures.txt' ---
line 39 skipped: radius must be >= 0
line 42 skipped: need 2 to 4 numbers
objects created: 19

--- Stored properties and computed attributes ---
  N      type      x      y      r      h      area      volume
-----
  1     Cone     2.00     3.00     1.60     5.00      8.04     13.40
  2     Cone     4.50     2.00     1.10     3.50      3.80      4.43
  3     Cone     3.00     5.50     2.80     2.00     24.63     16.42
  4     Cone     5.50     4.50     0.90     2.50      2.54      2.12
  5     Cone     14.00    12.00     2.40     8.00     18.10     48.25
  6     Cone     16.50    13.50     1.00     9.50      3.14      9.95
  7     Cone     15.00    15.50     1.80     6.50     10.18     22.05
  8     Cone     17.50    11.00     1.00     3.00      3.14      3.14
  9     Cone      9.00      8.00     3.00    10.00     28.27     94.25
 10     Cone     19.00      4.00     2.20     9.00     15.21     45.62
 11     Cone      6.00     14.00     1.50     4.50      7.07     10.60
 12     Cone      8.00     17.00     1.70     5.50      9.08     16.65
 13   Circle     11.00      3.00     1.40      -      6.16      -
 14   Circle     12.50     17.50     2.00      -     12.57      -

wasdm@ziliboba MINGW64 ~/cones
$
```

Скриншот 5. Запуск figures: разбор файла, таблица фигур, пропуск ошибочных строк

Вторая программа – figures\_list: те же фигуры складываются в двупольный список STL и упорядочиваются сначала по площади основания, потом по объёму. Порядки получаются разными, и это видно на конусе с широким низким основанием.

```

wasdm@ziliboba MINGW64 ~/cones
$ ./bin/figures_list | sed -n '52,62p;78,86p'
13   Cone      8.00   17.00   1.70   5.50      9.08      16.65
14   Cone     15.00   15.50   1.80   6.50     10.18     22.05
15   Circle    12.50   17.50   2.00   -        12.57     -
16   Cone     19.00   4.00    2.20   9.00     15.21     45.62
17   Cone     14.00   12.00   2.40   8.00     18.10     48.25
18   Cone      3.00    5.50    2.80   2.00     24.63     16.42
19   Cone      9.00    8.00    3.00   10.00    28.27     94.25
-----
19                                     total:    156.45    286.89

13   Cone      2.00    3.00    1.60    5.00      8.04     13.40
14   Cone      3.00    5.50    2.80    2.00     24.63     16.42
15   Cone      8.00   17.00    1.70    5.50      9.08     16.65
16   Cone     15.00   15.50    1.80    6.50     10.18     22.05
17   Cone     19.00    4.00    2.20    9.00     15.21     45.62
18   Cone     14.00   12.00    2.40    8.00     18.10     48.25
19   Cone      9.00    8.00    3.00   10.00    28.27     94.25
-----
19                                     total:    156.45    286.89

wasdm@ziliboba MINGW64 ~/cones
$ ./bin/figures_list | tail -4
-----
5                                     total:      84.32    239.49

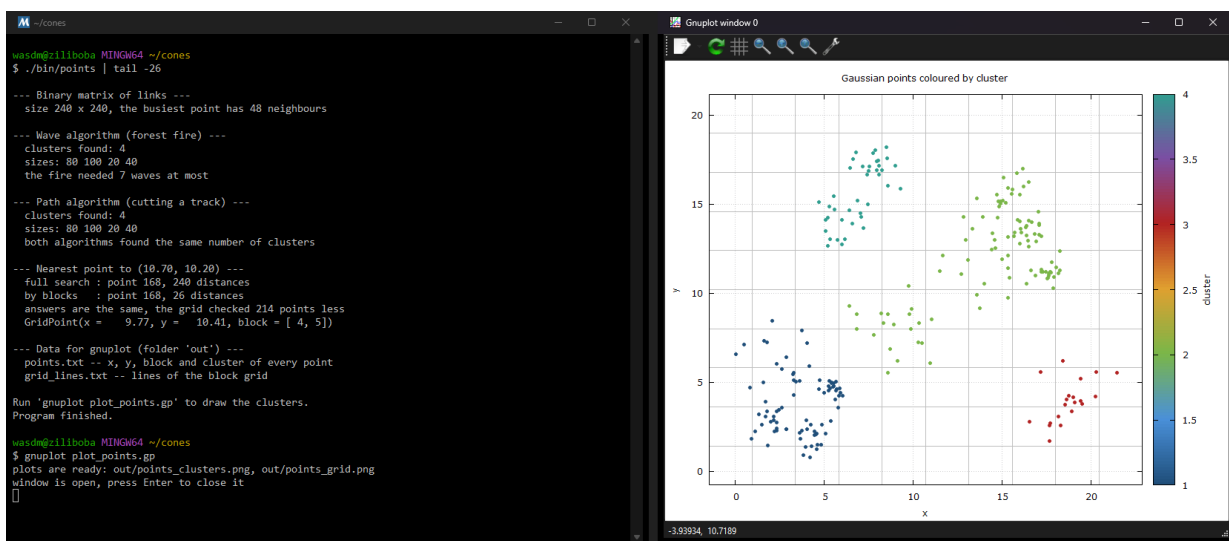
Program finished.

wasdm@ziliboba MINGW64 ~/cones
$

```

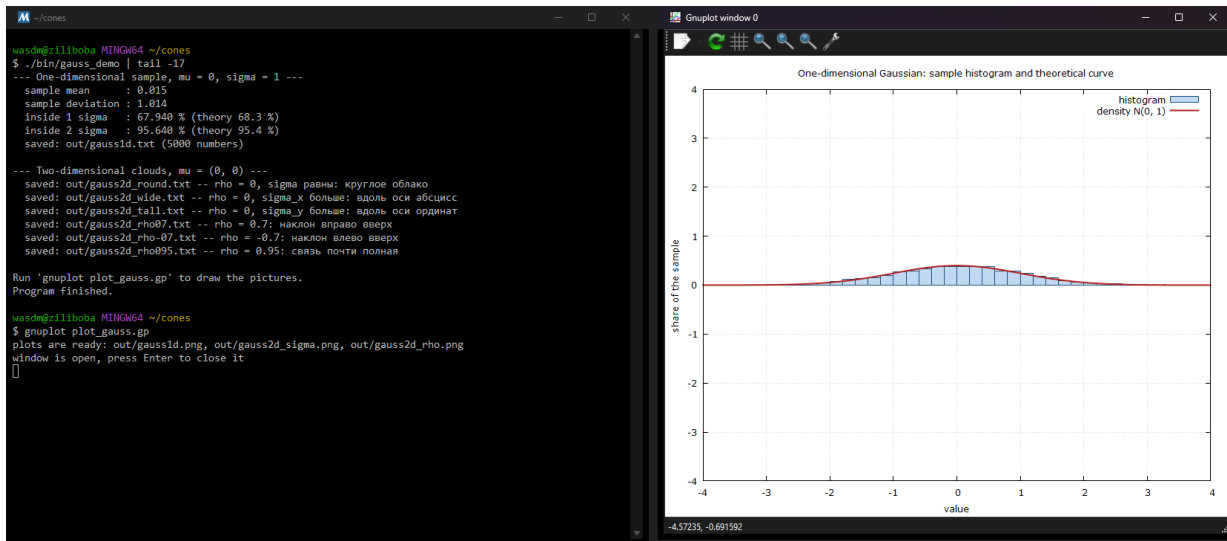
Скриншот 6. Запуск figures\_list: список STL и два порядка сортировки

Третья программа – points: вокруг каждого конуса она рассыпает точки, а рядом с терминалом снова видно окно gnuplot – на этот раз с раскрашенными кластерами. Подробнее: она рассыпает гауссовы точки, приписывает каждой номер блока сетки, подбирает порог по минимальному покрывающему дереву и ищет кластеры двумя способами. Оба дали 4 кластера размерами 80, 100, 20 и 40 точек, а поиск ближайшей точки по блокам обошёлся 26 вычислениями расстояния вместо 240 при полном переборе – ответ тот же.



Скриншот 7. Запуск points и окно gnuplot: порог, кластеры двумя алгоритмами, поиск по сетке

Четвёртая программа – `gauss_demo`: она проверяет сам датчик случайных чисел. По выборке из 5000 чисел получились среднее 0.015 и отклонение 1.014 при заданных 0 и 1, внутрь одной сигмы попало 67.9 процента значений, внутрь двух – 95.6 процента против теоретических 68.3 и 95.4.



Скриншот 8. Запуск `gauss_demo` и окно `gnuplot`: проверка датчика по правилам одной и двух сигм

## 6. Скрипты `gnuplot`

Рисунки строит не программа на C++, а `gnuplot` по её данным. Все три скрипта устроены одинаково: сначала задаётся терминал `pngcairo` (вывод в файл `png`), затем подписи и границы поля, потом по одной команде `plot` или `splot` на каждый рисунок, а в конце – терминал `qt`, то есть тот же рисунок в окне на экране, и команда `pause`, которая держит окно открытым.



```
~/cones
$ cat -n plot_figures.gp | sed -n '1,46p'
1 # plot_figures.gp -- visualisation of the figure field (top view and landscape)
2 # Автор: Горчак Дмитрий, 212
3 #
4 # Сначала запустить программу figures: она создаёт в папке out файлы
5 # plan_points.txt, plan_circles.txt, plan_cones.txt (вид сверху),
6 # field.txt (сетка высот) и range.gp (границы поля по x и y).
7 # Затем из папки cones:
8 #   gnuplot plot_figures.gp
9 # Готовые рисунки складываются в ту же папку out.
10 # Чтобы посмотреть картинку в окне, замените терминал на: set terminal qt persist
11
12 set terminal pngcairo size 1000,800 font 'Verdana,10'
13 set grid
14 set xlabel "x"
15 set ylabel "y"
16
17 # Границы поля посчитаны программой, а не заданы руками
18 load 'out/range.gp'
19
20 # 1. Вид сверху: конусы, круги и точки -- каждый своим стилем
21 set output "out/plan.png"
22 set title "Figures on a plane: top view"
23 set size ratio -1
24 set key top left
25 set style fill transparent solid 0.25 border lc rgb "#1f4e79"
26 plot "out/plan_cones.txt" using 1:2:3 with circles lc rgb "#4a90d9" title "cones", \
27      "out/plan_circles.txt" using 1:2:3 with circles lc rgb "#7ab648" title "circles", \
28      "out/plan_cones.txt" using 1:2 with points pt 7 ps 0.8 lc rgb "#b22222" notitle, \
29      "out/plan_points.txt" using 1:2 with points pt 7 ps 1.2 lc rgb "#333333" title "points", \
30      "out/plan_cones.txt" using 1:2:(sprintf("h=%.1f", $4)) with labels \
31      offset 0,1.2 font ",8" textcolor rgb "#404040" notitle
32 unset output
33
34 # 2. Карта высот: ландшафт конусов, вид строго сверху
35 set output "out/map.png"
36 set title "Height map z(x, y)"
37 set view map
38 set pm3d at b
39 set palette defined (0 "#eef3e6", 0.2 "#8fbf5a", 0.5 "#e0c878", 0.8 "#a8743a", 1 "#ffffff")
40 set cblabel "height"
41 unset key
42 splot "out/field.txt" using 1:2:3 with pm3d notitle
43 unset output
44
45 # 3. Ландшафт: поверхность z(x, y) над плоскостью
46 set output "out/landscape.png"

wasdm@ziliboba MINGW64 ~/cones
$
```

Скриншот 9. plot\_figures.gp, строки 1–46: вид сверху и карта высот

```
~/cones
wasdm@ziliboba MINGW64 ~/cones
$ cat -n plot_figures.gp | sed -n '47,83p'
47 set title "Cone landscape z(x, y)"
48 unset view
49 set view 55, 35, 1.0, 1.0
50 set size ratio 1
51 set xyplane at 0
52 set xlabel "h"
53 set pm3d
54 set hidden3d
55 splot "out/field.txt" using 1:2:3 with pm3d notitle
56 unset output
57
58 # 4. Фигуры точками: облако точек на поверхности каждой фигуры
59 set output "out/points3d.png"
60 set title "Figures drawn with points on their surface"
61 unset pm3d
62 unset hidden3d
63 set view 62, 30, 1.0, 1.0
64 splot "out/surface_points.txt" using 1:2:3 with points pt 7 ps 0.2 \
65     lc rgb "#1f4e79" notitle
66 unset output
67
68 print "plots are ready: out/plan.png, out/map.png, out/landscape.png, out/points3d.png"
69
70 # 5. То же поле в окне на экране: терминал qt вместо файла png
71 set terminal qt size 760,620 font 'Verdana,9'
72 set title "Figures on a plane: top view"
73 unset view
74 set size ratio -1
75 set key top left
76 plot "out/plan_cones.txt" using 1:2:3 with circles lc rgb "#4a90d9" title "cones", \
77     "out/plan_circles.txt" using 1:2:3 with circles lc rgb "#7ab648" title "circles", \
78     "out/plan_cones.txt" using 1:2 with points pt 7 ps 0.8 lc rgb "#b22222" notitle, \
79     "out/plan_points.txt" using 1:2 with points pt 7 ps 1.2 lc rgb "#333333" title "points", \
80     "out/plan_cones.txt" using 1:2:(sprintf("h=%.1f", $4)) with labels \
81     offset 0,1.2 font ",8" textcolor rgb "#404040" notitle
82
83 pause -1 "window is open, press Enter to close it"

wasdm@ziliboba MINGW64 ~/cones
$
```

Скриншот 10. plot\_figures.gp, строки 47–83: ландшафт, точки по поверхности и окно на экране

```
wasdm@ziliboba MINGW64 ~/cones
$ cat -n plot_points.gp
1 # plot_points.gp -- visualisation of the Gaussian points, the block grid and the clusters
2 # Автор: Горчак Дмитрий, 212
3 #
4 # Сначала запустить программу points: она создаёт в папке out файлы
5 # points.txt (x, y, номера блока и кластера) и grid_lines.txt (линии сетки).
6 # Затем из папки cones:
7 #   gnuplot plot_points.gp
8 # Готовые рисунки: out/points_clusters.png и out/points_grid.png.
9
10 set terminal pngcairo size 1000,800 font 'Verdana,10'
11 set grid
12 set xlabel "x"
13 set ylabel "y"
14 set size ratio -1
15 load 'out/range.gp'
16
17 # 1. Точки, раскрашенные по номеру кластера
18 set output "out/points_clusters.png"
19 set title "Gaussian points coloured by cluster"
20 unset key
21 set palette defined (0 "#1f4e79", 1 "#4a90d9", 2 "#7ab648", 3 "#e0a030", \
22                     4 "#b22222", 5 "#7b4ea3", 6 "#2f9e8f")
23 set cblabel "cluster"
24 plot "out/points.txt" using 1:2:5 with points pt 7 ps 0.7 palette notitle
25 unset output
26
27 # 2. Те же точки и сетка блоков: у каждой точки есть две координаты блока
28 set output "out/points_grid.png"
29 set title "Points and the grid of blocks"
30 unset colorbox
31 plot "out/grid_lines.txt" using 1:2 with lines lw 1 lc rgb "#c0c0c0" notitle, \
32       "out/points.txt" using 1:2:5 with points pt 7 ps 0.6 palette notitle
33 unset output
34
35 print "plots are ready: out/points_clusters.png, out/points_grid.png"
36
37 # 3. Те же кластеры в окне на экране: терминал qt вместо файла png
38 set terminal qt size 760,620 font 'Verdana,9'
39 set title "Gaussian points coloured by cluster"
40 set colorbox
41 plot "out/grid_lines.txt" using 1:2 with lines lw 1 lc rgb "#c0c0c0" notitle, \
42       "out/points.txt" using 1:2:5 with points pt 7 ps 0.7 palette notitle
43
44 pause -1 "window is open, press Enter to close it"

wasdm@ziliboba MINGW64 ~/cones
$
```

Скриншот 11. plot\_points.gp целиком: кластеры цветом и сетка блоков

```

M ~/cones
$ cat -n plot_gauss.gp | sed -n '1,46p'
 1 # plot_gauss.gp -- illustrations for the Gaussian distribution
 2 # Автор: Горчак Дмитрий, 212
 3 #
 4 # Сначала запустить программу gauss_demo: она создаёт в папке out файл
 5 # gauss1d.txt и шесть файлов gauss2d_*.txt с двумерными облаками.
 6 # Затем из папки cones:
 7 # gnuplot plot_gauss.gp
 8 # Готовые рисунки: out/gauss1d.png, out/gauss2d_sigma.png, out/gauss2d_rho.png.
 9
10 set terminal pngcairo size 1000,700 font 'Verdana,10'
11
12 # 1. Одномерный случай: гистограмма выборки и теоретическая кривая
13 set output "out/gauss1d.png"
14 set title "One-dimensional Gaussian: sample histogram and theoretical curve"
15 set xlabel "value"
16 set ylabel "share of the sample"
17 set grid
18 set xrange [-4:4]
19 set key top right
20 set boxwidth 0.2
21 set style fill transparent solid 0.35 border lc rgb "#1f4e79"
22
23 bin = 0.2                # ширина столбца гистограммы
24 n = 5000.0              # размер выборки
25 mu = 0.0
26 sigma = 1.0
27 density(x) = exp(-(x - mu)**2 / (2 * sigma**2)) / (sigma * sqrt(2 * pi))
28
29 plot "out/gauss1d.txt" using (bin * floor($1 / bin) + bin / 2):(1.0 / (n * bin)) \
30     smooth freq with boxes lc rgb "#4a90d9" title "histogram", \
31     density(x) with lines lw 2 lc rgb "#b22222" title "density N(0, 1)"
32 unset output
33
34 # 2. Двумерный случай без связи координат: форму задают только разбросы
35 set terminal pngcairo size 1200,440 font 'Verdana,10'
36 set output "out/gauss2d_sigma.png"
37 set multiplot layout 1,3 title "Two-dimensional Gaussian, rho = 0: the shape is set by sigma"
38 set size ratio -1
39 set xrange [-4:4]
40 set yrange [-4:4]
41 set xlabel "x"
42 set ylabel "y"
43 unset key
44
45 set title "sigma_x = sigma_y = 1: round cloud"
46 plot "out/gauss2d_round.txt" using 1:2 with points pt 7 ps 0.3 lc rgb "#1f4e79" notitle

wasdm@ziliboba MINGW64 ~/cones
$

```

Скриншот 12. plot\_gauss.gp, строки 1–46: гистограмма с теоретической кривой и облака при  $\rho = 0$

```
~/cones
wasdm@ziliboba MINGW64 ~/cones
$ cat -n plot_gauss.gp | sed -n '47,87p'
47
48 set title "sigma_x = 2, sigma_y = 0.6: along the x axis"
49 plot "out/gauss2d_wide.txt" using 1:2 with points pt 7 ps 0.3 lc rgb "#2e7d32" notitle
50
51 set title "sigma_x = 0.6, sigma_y = 2: along the y axis"
52 plot "out/gauss2d_tall.txt" using 1:2 with points pt 7 ps 0.3 lc rgb "#6a3d9a" notitle
53
54 unset multiplot
55 unset output
56
57 # 3. Двумерный случай со связью координат: разбросы равны, меняется rho
58 set output "out/gauss2d_rho.png"
59 set multiplot layout 1,3 title "Two-dimensional Gaussian, sigma_x = sigma_y = 1: the shape is se
t by rho"
60
61 set title "rho = 0.7: tilted to the right"
62 plot "out/gauss2d_rho07.txt" using 1:2 with points pt 7 ps 0.3 lc rgb "#b22222" notitle
63
64 set title "rho = -0.7: tilted to the left"
65 plot "out/gauss2d_rho-07.txt" using 1:2 with points pt 7 ps 0.3 lc rgb "#c46210" notitle
66
67 set title "rho = 0.95: almost a straight line"
68 plot "out/gauss2d_rho095.txt" using 1:2 with points pt 7 ps 0.3 lc rgb "#8b0000" notitle
69
70 unset multiplot
71 unset output
72
73 print "plots are ready: out/gauss1d.png, out/gauss2d_sigma.png, out/gauss2d_rho.png"
74
75 # 4. Та же гистограмма в окне на экране: терминал qt вместо файла png
76 set terminal qt size 760,620 font 'Verdana,9'
77 set title "One-dimensional Gaussian: sample histogram and theoretical curve"
78 set size noratio
79 set xlabel "value"
80 set ylabel "share of the sample"
81 set xrange [-4:4]
82 set key top right
83 plot "out/gauss1d.txt" using (bin * floor($1 / bin) + bin / 2):(1.0 / (n * bin)) \
84     smooth freq with boxes lc rgb "#4a90d9" title "histogram", \
85     density(x) with lines lw 2 lc rgb "#b22222" title "density N(0, 1)"
86
87 pause -1 "window is open, press Enter to close it"

wasdm@ziliboba MINGW64 ~/cones
$
```

Скриншот 13. plot\_gauss.gp, строки 47–87: облака при разных rho и окно на экране

Ни одно число границ в скриптах не задано руками: строка load 'out/range.gp' подключает файл, который пишет сама программа. Гистограмма строится приёмом smooth freq: gnuplot сам считает, сколько чисел попало в каждую полосу ширины bin, а деление на n \* bin переводит счётчик в долю выборки, чтобы столбики совпали по высоте с теоретической кривой плотности.

## 7. Результат: поле фигур

Вид сверху: конусы, круги и точки – каждый тип своим стилем, у конусов подписана высота. Видно, что конусы собраны в две группы и несколько одиночных фигур – это заготовка для будущей задачи о кластерах.

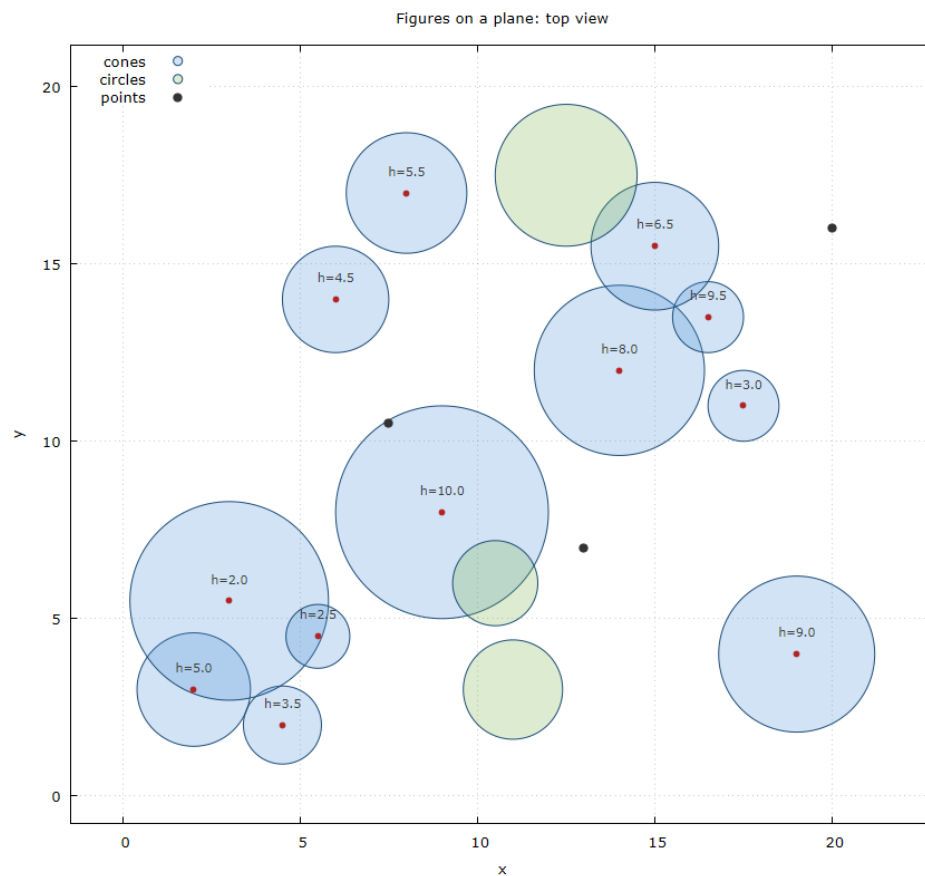


Рисунок 1. Вид сверху: конусы, круги и точки на плоскости

Ландшафт: высота поверхности в точке равна максимуму по всем конусам, поэтому там, где основания пересекаются, виден более высокий конус. Сетку 121 x 121 узел считает программа и сохраняет в файл, gnuplot только рисует.

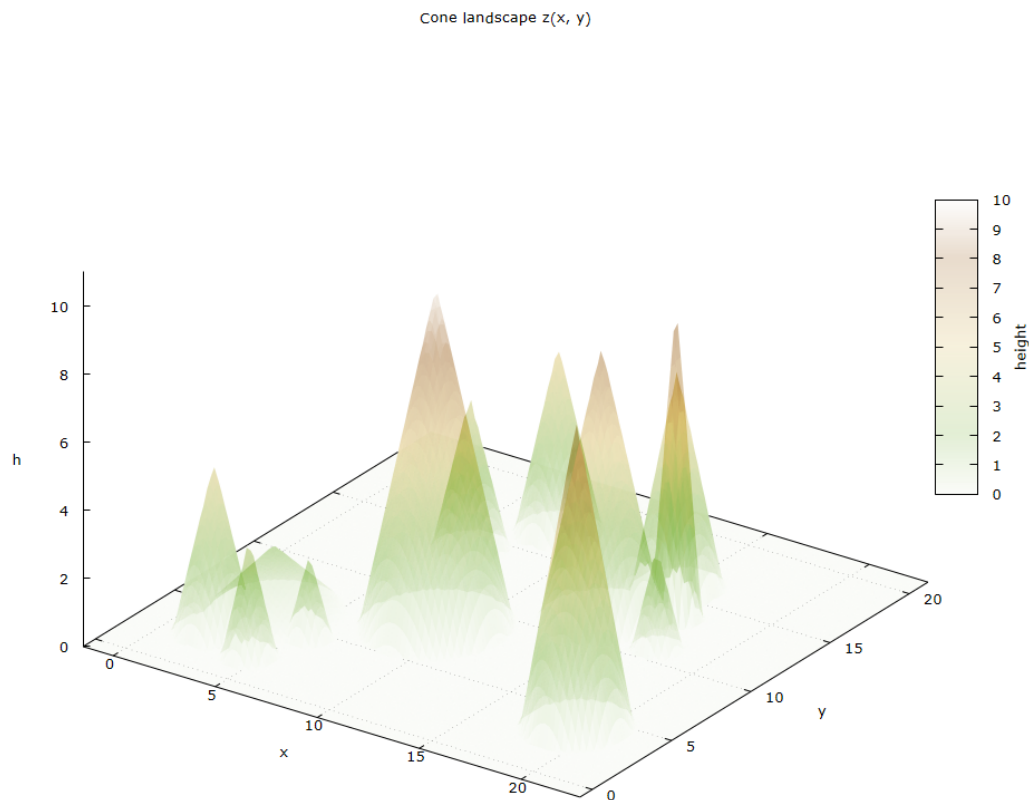


Рисунок 2. Ландшафт  $z(x, y)$ : поверхность конусов над плоскостью

Третий рисунок показывает фигуры точками на их поверхности. Точки расставлены не перебором пространства, а по уравнению поверхности: на высоте  $z$  радиус сечения конуса равен  $r * (1 - z / h)$ , и по этой окружности точки ставятся через равные углы. Уравнение каждой фигуры программа дополнительно записывает в файл `out/equations.txt`.

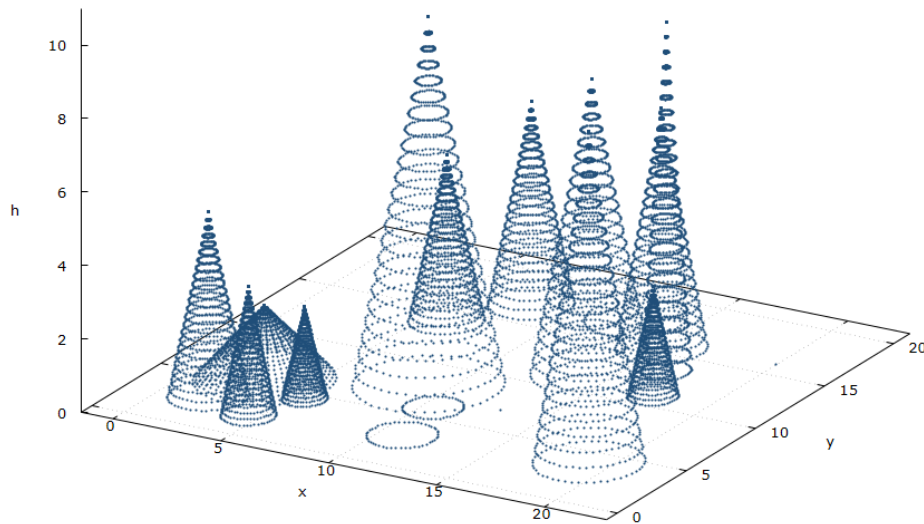


Рисунок 3. Фигуры, нарисованные точками по поверхности

## 8. Результат: точки, сетка и кластеры

Вокруг каждого конуса рассыпано по двадцать точек с гауссовым разбросом, равным половине радиуса основания: при таком разбросе почти все точки остаются внутри основания. Программа ничего не знает о том, какая точка от какого конуса, и восстанавливает группы только по расстояниям.



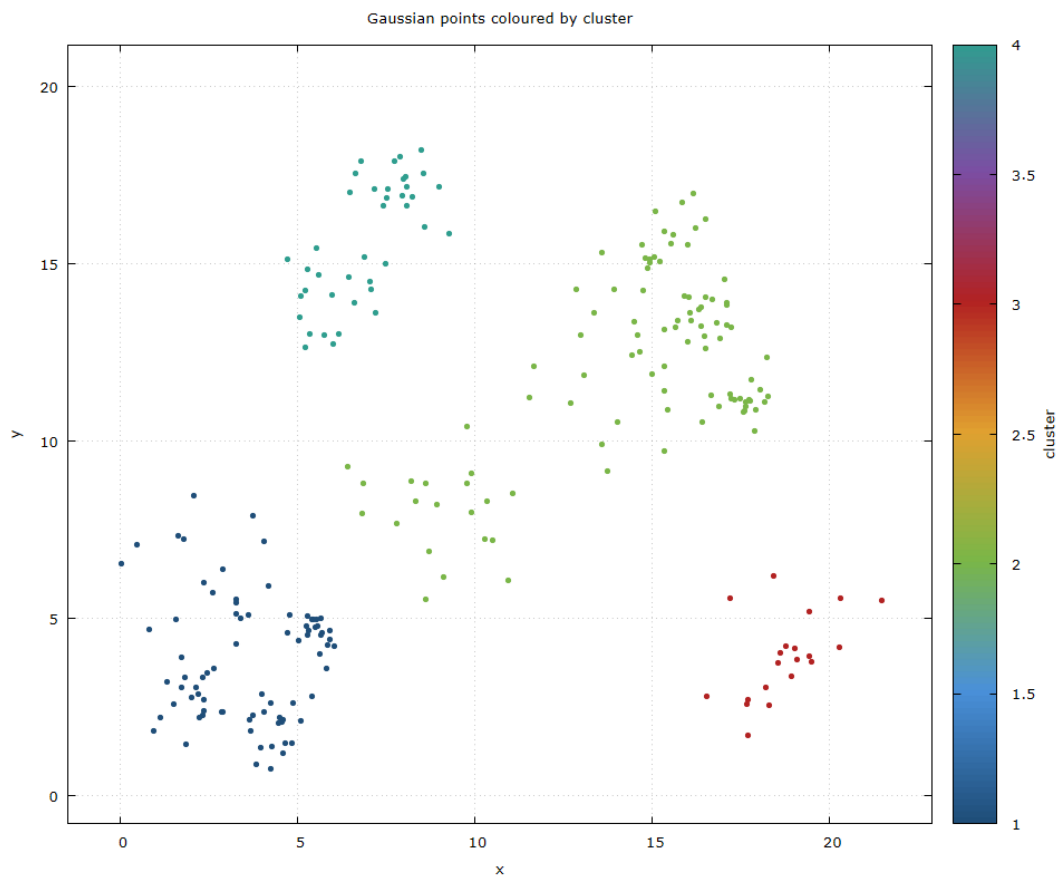


Рисунок 4. Кластеры, найденные по порогу 2.39: цвет – номер связной компоненты

Сетка блоков нужна, чтобы не перебирать всё облако при поиске соседей: по координатам точки сразу считается номер её блока, и просматриваются только сам блок и ближайшие к нему.

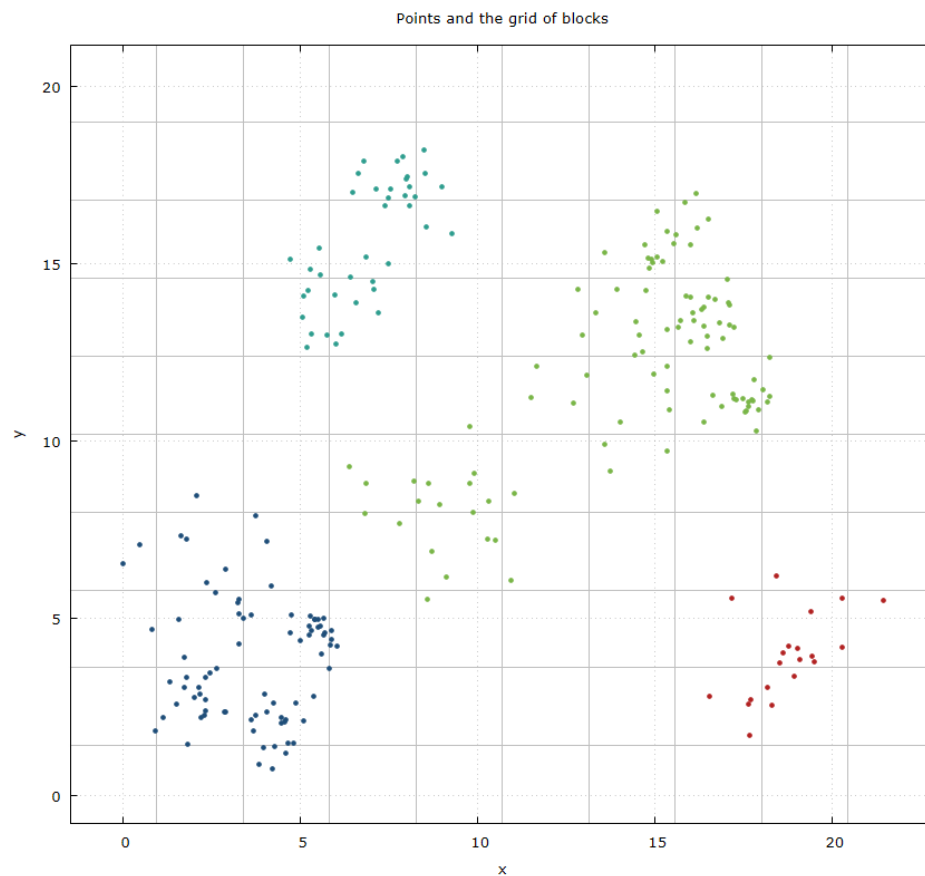


Рисунок 5. Сетка 10 x 10 блоков и распределение точек по ней

Отдельно проверен датчик случайных чисел. Гистограмма выборки ложится на теоретическую кривую плотности. Двумерных облаков построено шесть, чтобы разделить два действия: разбросы по осям растягивают облако вдоль осей координат, а коэффициент связи координат его поворачивает.

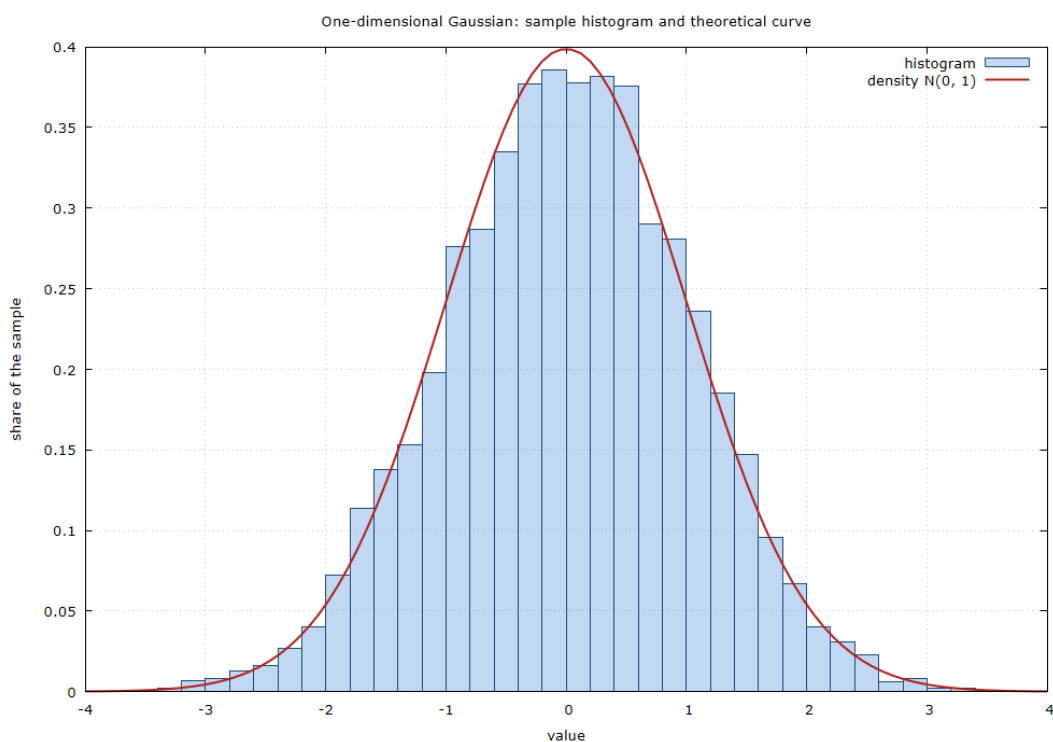


Рисунок 6. Гистограмма выборки и теоретическая кривая  $N(0, 1)$

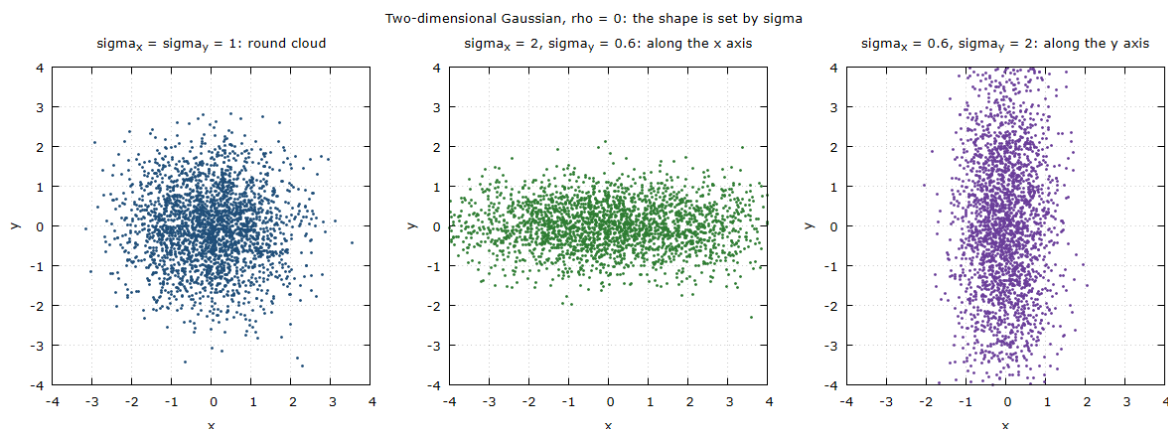


Рисунок 7. Связи нет: круглое облако, облако вдоль оси абсцисс и вдоль оси ординат

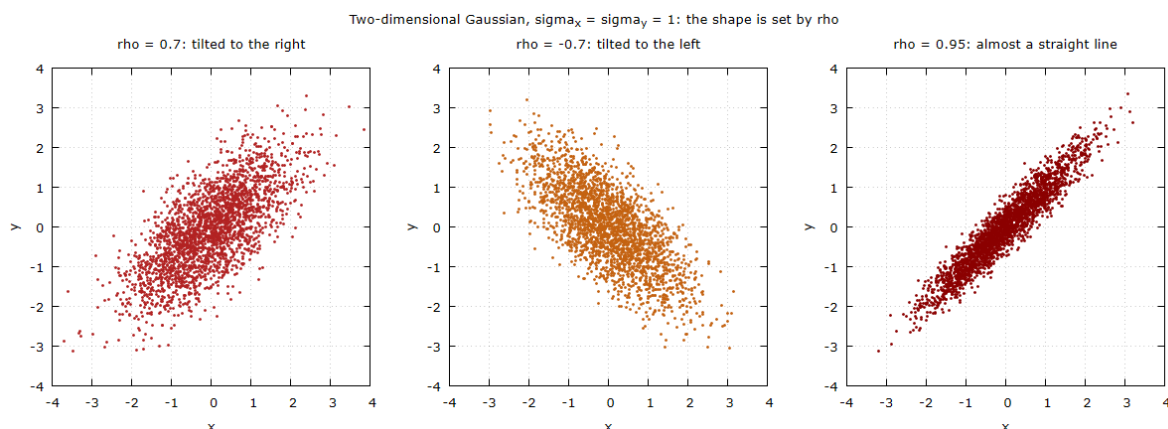


Рисунок 8. Разбросы равны, меняется только rho: 0.7, -0.7 и 0.95

## 9. Состав репозитория

| Файл                            | Что внутри                                                 |
|---------------------------------|------------------------------------------------------------|
| point.h, point.cpp              | класс Point: координаты, расстояние между точками          |
| circle.h, circle.cpp            | класс Circle: радиус и площадь как метод                   |
| cone.h, cone.cpp                | класс Cone: высота, объём, высота поверхности              |
| figures_io.h,<br>figures_io.cpp | чтение файла, таблица, данные для gnuplot                  |
| grid_point.h, .cpp              | класс GridPoint: точка с номером блока сетки               |
| grid.h, grid.cpp                | сетка блоков и поиск ближайшей точки по блокам             |
| gauss.h, gauss.cpp              | класс Gauss: датчик нормальных чисел, одно и два измерения |
| cluster.h, cluster.cpp          | матрица расстояний, порог, волновой алгоритм и просека     |
| main.cpp                        | программа figures: таблица фигур и подготовка рисунков     |
| list_demo.cpp                   | программа figures_list: двунаправленный список STL         |
| points_demo.cpp                 | программа points: гауссовы точки, сетка, кластеры          |
| gauss_demo.cpp                  | программа gauss_demo: проверка датчика случайных чисел     |
| figures.txt                     | исходные данные: от двух до четырёх чисел в строке         |
| plot_figures.gp                 | gnuplot: четыре рисунка по фигурам и окно на экране        |
| plot_points.gp                  | gnuplot: кластеры, сетка блоков и окно на экране           |
| plot_gauss.gp                   | gnuplot: гистограмма, облака и окно на экране              |
| build.sh, make_txt.sh           | сборка и копии исходников в .txt                           |
| README.md                       | описание задачи, состава и порядка запуска                 |

|         |                                          |
|---------|------------------------------------------|
| essays/ | эссе, этот отчёт, скриншоты и рисунки    |
| txt/    | копии всех исходников с расширением .txt |

Все исходники лежат в одной папке ~/cones без вложенных каталогов; отдельные папки заведены только под материалы отчёта (essays) и под копии исходников в текстовом виде (txt). Собирается всё одной командой `bash build.sh`.

## 10. Репозиторий в облаке

Локальная история целиком выложена на GitHub вместе с обеими метками:

<https://github.com/crypto-dimitri-hash/212cppcryptodimitri>

Перенос занял две команды: `git remote add` задаёт адрес копии под именем `origin`, `git push -u origin main --tags` отправляет туда ветку и метки. Повседневные команды от этого не изменились – фиксация по-прежнему локальная, а `push` отправляет уже готовые коммиты.

Вход сделан по ключу SSH, а не по паролю: `ssh-keygen` создаёт пару из секретного и открытого ключей, открытый добавляется в настройках GitHub, секретный остаётся на компьютере. После этого пароль не запрашивается, а по сети передаётся не сам ключ, а подпись, сделанная им.

## 11. Что дальше

- 1) ускорить построение матрицы связей: сейчас она строится полным перебором пар, хотя размер блока можно взять равным порогу и проверять только соседние блоки;
- 2) сравнить порог, подобранный по покрывающему дереву, с порогами, заданными вручную, и посмотреть, при каком значении кластеры начинают сливаться.